본 저작물은 「저작권법」 제25조(학교교육 목적 등에의 이용)에 따라
적법하게 이용되고 있습니다.
이용 범위를 초과한 사용은 저작권법에 저촉될 수 있으므로,
저작물의 재복제 및 수업 목적 외 사용을 금지합니다.

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

아이디어를 현실로 - **바이브 코딩**

Firebase 소개와 연동

BaaS와 Firebase 소개

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 학습 목표

1

BaaS 개념 이해하기

Backend as a Service가 무엇인지 배웁니다

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 학습 목표

1. **BaaS 개념 이해하기**
   Backend as a Service가 무엇인지 배웁니다

2. **Firestore 데이터 구조 파악하기**
   컬렉션, 문서, 서브컬렉션 개념 이해

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 학습 목표

1. **BaaS 개념 이해하기**
   Backend as a Service가 무엇인지 배웁니다

2. **Firestore 데이터 구조 파악하기**
   컬렉션, 문서, 서브컬렉션 개념 이해

3. **Firebase 프로젝트 생성하기**
   직접 설정하고 Next.js와 연동

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 학습 목표

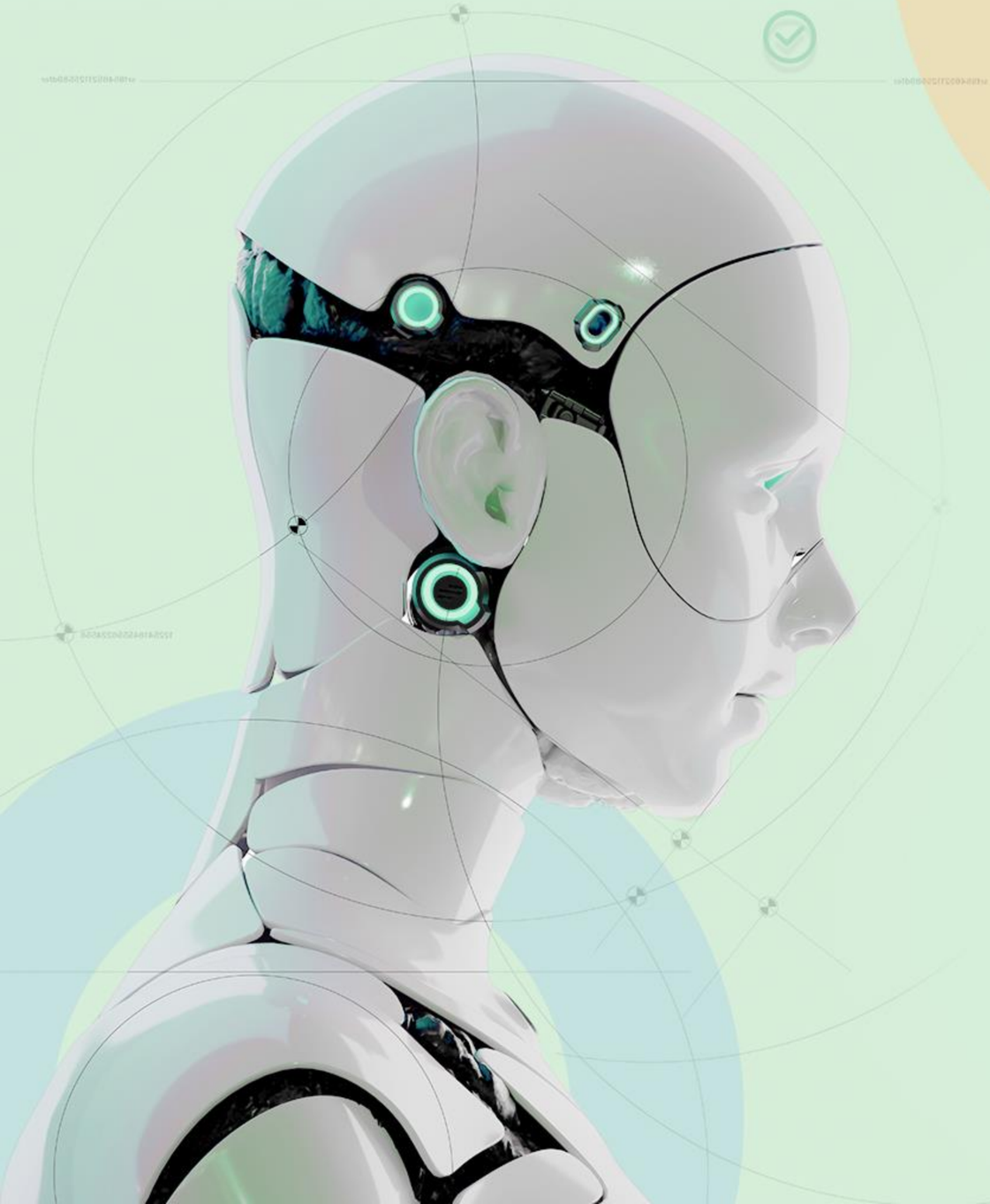
1. **BaaS 개념 이해하기**
   Backend as a Service가 무엇인지 배웁니다

2. **Firestore 데이터 구조 파악하기**
   컬렉션, 문서, 서브컬렉션 개념 이해

3. **Firebase 프로젝트 생성하기**
   직접 설정하고 Next.js와 연동

4. **데이터 영구 저장하기**
   Create/Read로 새로고침 문제 해결

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.



CONTENTS

01 BaaS와 Firebase 소개

02 Firestore 데이터 구조와 설계

03 실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

마이링크 프로젝트 진행 상황

40%

지난주

임시 저장

50%

이번주

영구 저장!

60%

다음주

CRUD 완성

임시 저장에서 영구 저장으로! 🚀 오늘이 중요한 분기점이에요

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

🚨 새로고침하면 사라져요!

문제 상황(7주차 결과)

1. 링크 추가 → 목록에 나타남 ✅
2. 페이지 새로고침(F5) 🔄
3. **추가한 링크가 다 사라짐! 😱**

왜 이런 일이 생길까요?

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

왜 사라질까?

현재 상황

컴퓨터 메모리(RAM)에만 저장

메모리 = 임시 저장소
페이지 닫으면 초기화됨

마치 메모지에 적어둔 것과 같아요

→

필요한 것

영구 저장소 = 데이터베이스!

데이터베이스 = 금고
닫아도 데이터 유지

서버에 안전하게 보관!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 해결책: Firebase

✓ **인터넷 서버에 데이터 저장**
구글 클라우드에 안전하게 보관

✓ **새로고침해도 안 사라짐!**
페이지 닫았다 열어도 그대로

✓ **다른 기기에서도 접근 가능**
핸드폰, 태블릿 어디서든!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

BaaS란?(1/2)

BaaS(Backend as a Service)

서버 개발·운영을 대신해주는 클라우드 서비스

전통 개발 방식

- 프론트+백엔드 모두 개발
- 서버 설치, DB 설정, 보안...
- 전문 지식 필요, 시간 많이 소요

?

🤔 이런 문제를 어떻게 해결할 수 있을까요?

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

BaaS란?(2/2)

BaaS(Backend as a Service)

서버 개발·운영을 대신해주는 클라우드 서비스

전통 개발 방식

- 프론트+백엔드 모두 개발
- 서버 설치, DB 설정, 보안...
- 전문 지식 필요, 시간 많이 소요

BaaS 방식 ✨

- 프론트엔드만 개발하면 됨!
- 서버·DB·보안은 BaaS가 처리
- "데이터 저장해줘"만 요청하면 끝

대표 BaaS 서비스

Firebase(구글) · Supabase · AWS Amplify

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

전통 백엔드 vs BaaS 비교

구분	전통 백엔드	BaaS(Firebase)
서버	직접 설치·관리	자동 제공 ✓
DB	직접 설정·운영	Firestore 제공 ✓
보안	직접 구현	기본 제공 ✓
확장성	서버 증설 필요	자동 확장 ✓
비용	항상 발생	무료 OK ✓
난이도	높음(전문 지식)	낮음(비전공자 OK) ✓

결론: "서버? 구글이 알아서 해줄게" 🎉

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Firebase 소개

Firebase = 구글이 제공하는 BaaS 플랫폼
앱 개발에 필요한 거의 모든 백엔드 서비스 제공

주요 기능

Firestore
NoSQL 데이터베이스
오늘 배울 내용!

Authentication
회원가입·로그인
11주차 예정

Hosting
웹 배포
Vercel 대안

그 외
Storage, Functions
필요시 확장

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

왜 Firebase를 선택했나?

- ✅ 무료 플랜으로 충분 — 학습용에 완벽
- ✅ 바로 시작 가능 — 구글 계정만 있으면 OK
- ✅ Next.js와 궁합 좋음 — 우리 프로젝트에 딱!
- ✅ 문서화 잘 되어 있음 — 막히면 검색하면 끝
- ✅ 구글 생태계 시너지! — Gemini, Cloud와 찰떡

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Firebase 무료 플랜 할당량

1 GiB	50K	20K	20K
저장 용량	읽기/일	쓰기/일	삭제/일
텍스트면 충분	하루 5만 번	하루 2만 번	하루 2만 번

리전 선택 팁

- 어떤 리전이든 무료 사용 가능
- 가까운 리전 = 빠른 응답
- 권장: asia-northeast3(서울)

학습용으로 충분한가?

✅ 마이링크 프로젝트: 충분!
✅ 테스트 · 개발: 전혀 문제없음
⚠️ 실서비스: 사용자 늘면 유료 검토

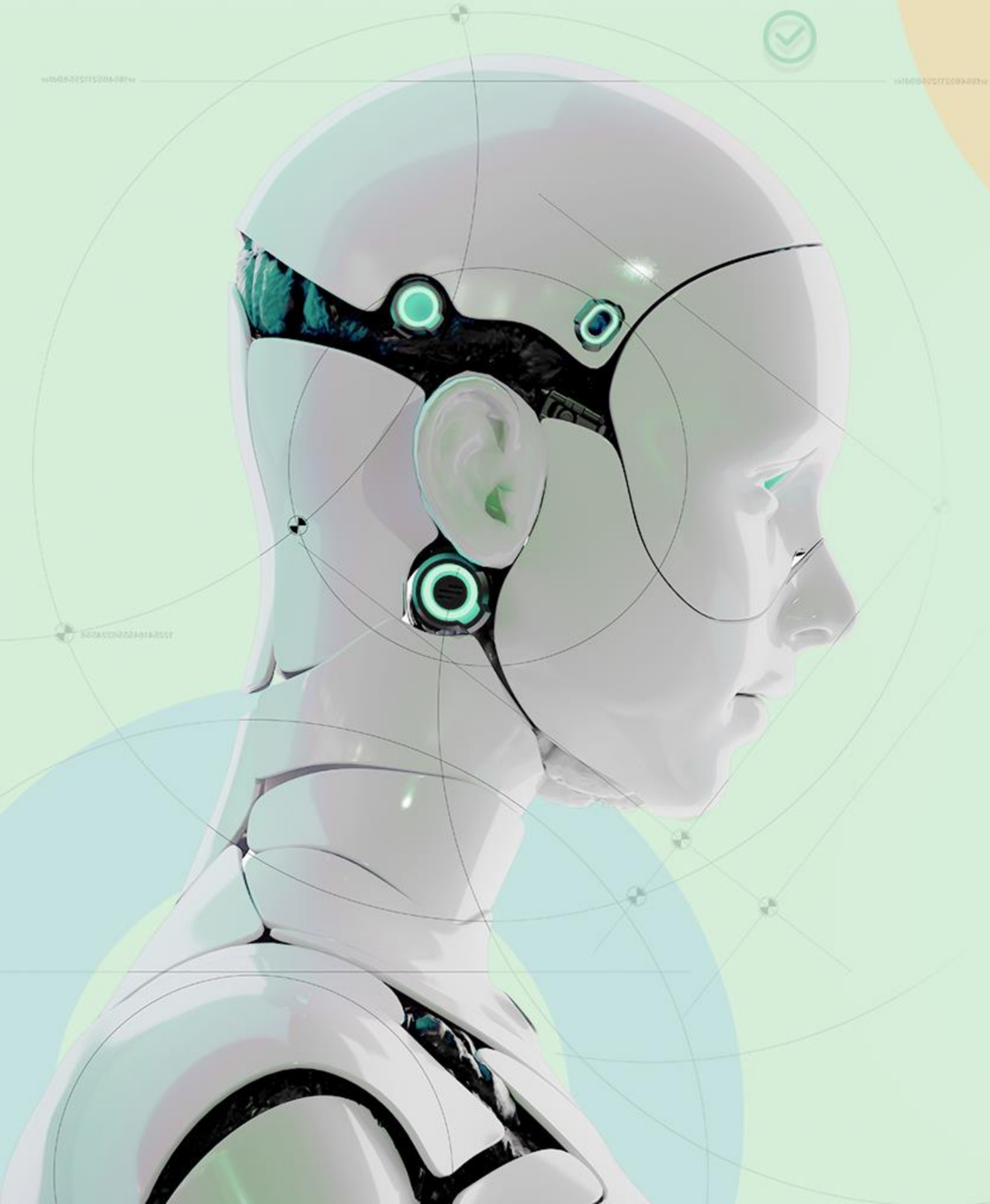
※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

아이디어를 현실로 - **바이브 코딩**

Firebase 소개와 연동

Firestore 데이터 구조와 설계

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.



CONTENTS

01 BaaS와 Firebase 소개

02 Firestore 데이터 구조와 설계

03 실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

2차시 시작

Firestore 데이터 구조와 설계

📚 지난 차시에서 배운 것

- BaaS: 서버 관리 불필요, 클라우드가 알아서
- Firebase: 구글 제공 BaaS 플랫폼
- 무료 플랜: 학습용으로 충분한 할당량

🎯 이번 차시 목표

- Firestore가 뭔지 이해하기(NoSQL)
- 컬렉션과 문서 개념 파악(폴더-파일)
- 서브컬렉션 이해하기(폴더 속 폴더)
- 마이링크 데이터 구조 설계하기

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Firestore란?

Firestore(Cloud Firestore)
Firebase의 NoSQL 클라우드 데이터베이스

특징

NoSQL — SQL(표 형식)이 아닌 유연한 구조
클라우드 저장 — 인터넷 서버에 영구 저장
확장성 — 사용자 늘어나도 자동 확장
보안 규칙 — 누가 읽고 쓸 수 있는지 설정

SQL vs NoSQL 간단 비교

SQL(관계형 DB)
엑셀처럼 표 형식, 고정된 구조
NoSQL(Firestore) ✨
"키: 값" 형태로 유연한 구조, 자유로운 형식

우리가 쓸 이유: 복잡한 표 설계 불필요, 비전공자도 쉽게 사용

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

SQL vs NoSQL 비교

구분	SQL(관계형 DB)	NoSQL(Firestore)
구조	표 형식(행, 열)	키-값 형식(유연)
스키마	고정(미리 정의)	자유(언제든 변경)
비유	엑셀 스프레드시트	폴더-파일 시스템
데이터 추가	열 구조 맞춰야 함	자유롭게 추가 가능
난이도	높음(SQL 문법 필요)	낮음(직관적) ✓

SQL: 은행 시스템 같은 복잡한 관계 필요할 때

마이링크는 NoSQL(Firestore)가 적합! 🎯

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

컬렉션과 문서 개념

폴더-파일 비유

```
📁 users(컬렉션 = 폴더)
├── 📄 user001(문서 = 파일)
│   ├── name: "홍길동"
│   ├── email: "hong@..."
│   └── age: 25
└── 📄 user002(문서)
    ├── name: "김철수"
    └── email: "kim@..."
```

컬렉션(Collection)

폴더처럼 문서들을 담는 그룹

예시:
users, links, posts

💡 이름은 복수형 권장

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

컬렉션과 문서 개념

문서(Document)

파일처럼 실제 데이터를 담는 단위

"키: 값" 쌍으로 구성
name: "홍길동"
email: "hong@example.com"

💡 각 문서는 고유 ID를 가짐

핵심 규칙

컬렉션 안에는?
문서만!

문서 안에는?
데이터 +(하위 컬렉션)

다음: 하위 컬렉션이 뭔지 알아볼게요!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

서브컬렉션 개념

폴더 속 폴더 비유

```
📁 users(컬렉션)
└── 📄 user001(문서)
    ├── name: "홍길동"
    ├── email: "hong@..."
    └── 📁 links(서브컬렉션)
        ├ 📄 link001
        │  ├ title: "인스타"
        │  └ url: "instagram.com/hong"
        └ 📄 link002 ...
```

서브컬렉션(Subcollection)

문서 안에 있는 또 다른 컬렉션

계층 구조 표현

사용자 → 링크들

경로 예시:

users/user001/links

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

왜 서브컬렉션을 쓸까?

데이터 관계 명확
"이 링크는 이 사용자 것"
소속이 명확해요

쿼리 효율
특정 사용자 링크만 조회
빠르고 정확해요

보안 규칙 쉬움
"본인 링크만 수정 가능"
12주차에 배워요!

이제 마이링크 프로젝트에 어떻게 적용하는지 볼게요!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

마이링크 데이터 구조 설계

현재(9주차): 서브컬렉션 방식

```
📁 users
└── 📄 anonymous(임시 사용자)
    └── 📁 links(서브컬렉션)
        ├ 📄 link001
        │  ├ title: "인스타그램"
        │  ├ url: "instagram.com/..."
        │  ├ createdAt: 날짜
        │  └ updatedAt: 날짜
        └ 📄 link002 ...
```

왜 anonymous?

아직 로그인 기능이 없어요!

지금은
모든 사용자가 같은 데이터 공유

11주차 이후
로그인 → 개인별 데이터 분리

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

데이터 경로 이해하기

현재 사용할 경로

users/anonymous/links/{linkId}

링크 추가: links 컬렉션에 문서 추가
링크 조회: links 컬렉션에서 모두 읽기

나중에 바뀔 경로(11주차+)

users/{userId}/links/{linkId}

userId: 로그인한 사용자 고유 ID
각자 자기만의 링크를 갖게 돼요!

지금은 anonymous로 연습! 나중에 쉽게 바꿀 수 있어요 👍

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

보안 규칙 개요

보안 규칙(Security Rules)이란?
Firestore 데이터를 누가 읽고 쓸 수 있는지 정하는 규칙

오늘(9주차): 테스트 모드

- 모든 사람이 읽고 쓸 수 있음(임시)
- 학습용으로만 사용(실서비스 X)
- ⚠️ 60일 후 자동 만료

```
// 테스트 모드(모든 접근 허용)
allow read, write: if true;
```

나중(12주차): 실제 보안 규칙

- 본인 데이터만 수정 가능
- 공개 데이터는 모두 읽기 가능
- 로그인한 사용자만 쓰기 가능

```
// 본인 데이터만 접근 가능
allow write: if
    request.auth.uid == userId;
```

지금은 걱정하지 마세요! 테스트 모드로 충분해요. 12주차에 제대로 배워요 👍

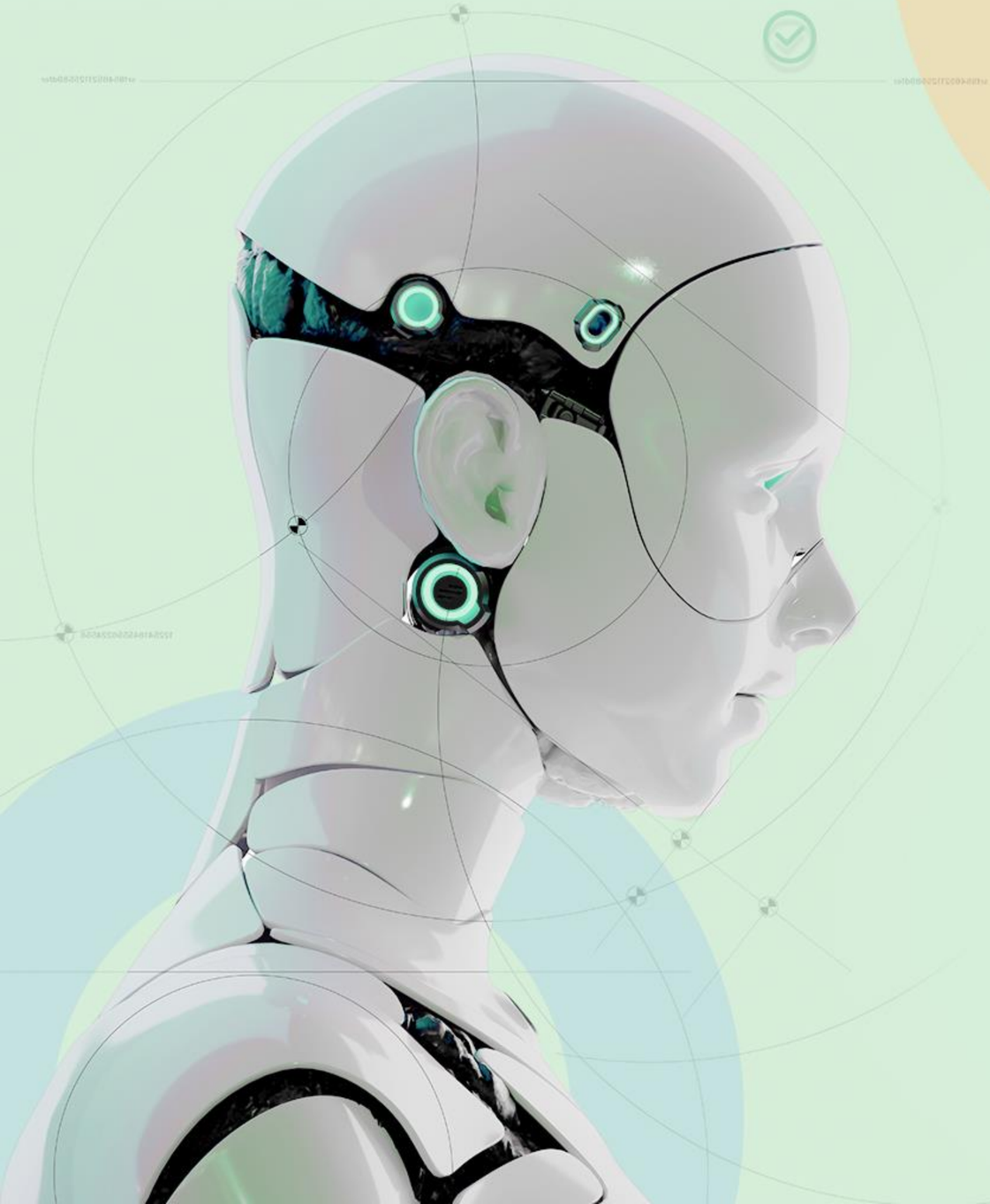
※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

아이디어를 현실로 - **바이브 코딩**

Firebase 소개와 연동

실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.



CONTENTS

01 BaaS와 Firebase 소개

02 Firestore 데이터 구조와 설계

03 실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

지난 시간 복습

실습과 마무리

📚 지난 차시에서 배운 것

- Firestore: NoSQL 데이터베이스
- 컬렉션-문서-서브컬렉션 구조
- 마이링크: users/anonymous/links/{linkId}
- 보안: 지금은 테스트 모드

🎯 이번 차시 목표

- Firebase 프로젝트 생성 및 설정
- Firestore 활성화(테스트 모드)
- Firebase SDK 설치 및 연동
- 링크 저장(Create) 및 불러오기(Read)
- 새로고침해도 데이터 유지! 🎉

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Firebase 프로젝트 생성(1/2)

1

Firebase 콘솔 접속

- console.firebase.google.com
- 구글 계정 로그인

2

프로젝트 생성

- "프로젝트 추가" 클릭
- 이름: mylink-project
- Analytics: 사용 안 함

3

?

4

?

콘솔에 접속하고 프로젝트를 생성한 다음은...?

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Firebase 프로젝트 생성(2/2)

1

Firebase 콘솔 접속

- console.firebase.google.com
- 구글 계정 로그인

2

프로젝트 생성

- "프로젝트 추가" 클릭
- 이름: mylink-project
- Analytics: 사용 안 함

3

Firestore 활성화

- "Firestore Database"
- 테스트 모드 선택
- 리전: asia-northeast3

4

웹 앱 등록

- 프로젝트 설정
- "웹 앱 추가" 클릭
- 설정 정보 복사

4단계 완료하면 Firebase 준비 끝! 🎉

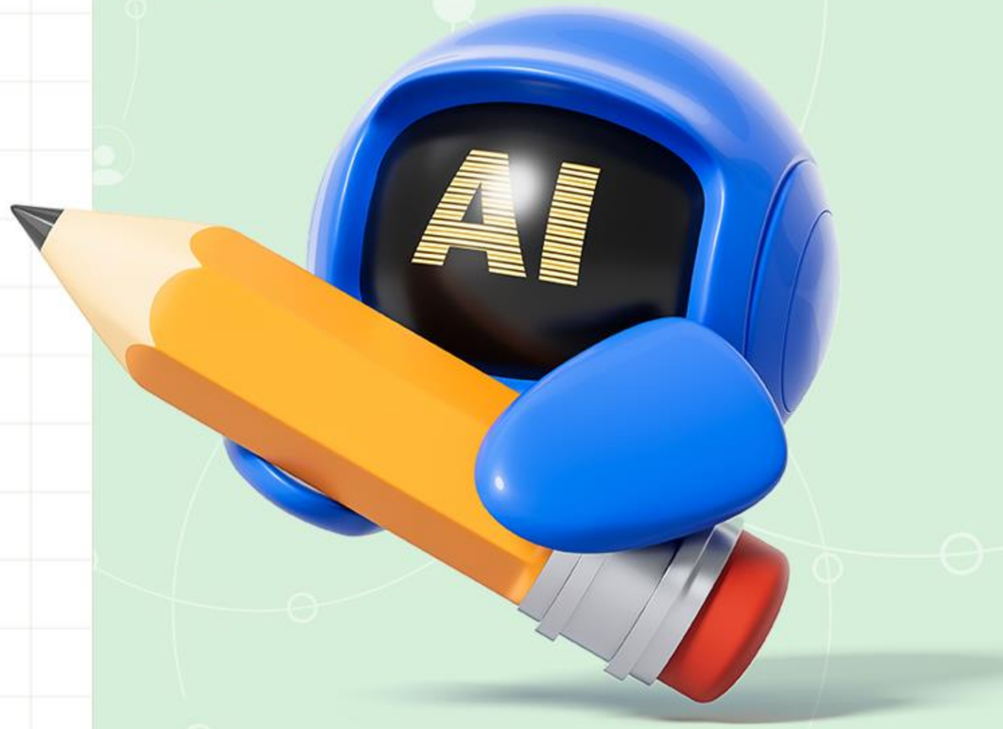
이론은 충분히! 이제 손으로 배워봐요

직접 해보기

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 실습 체크리스트

- [ ] Firebase 프로젝트 생성
- [ ] Firestore 생성 - 테스트 모드
- [ ] Firebase SDK 설치 및 설정
- [ ] 링크 저장 기능(Create)
- [ ] 링크 불러오기(Read)
- [ ] 새로고침 후 유지 확인



※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

실습 시연 흐름

시연 흐름

1

Firebase 콘솔 설정

로그인 → 프로젝트 생성
→ Firestore 활성화

2

AI로 SDK 연동

"Firebase 연동해줘" 프롬프트

3

CRUD 구현

"링크 저장을 Firestore로"
프롬프트

4

결과 확인

새로고침 → 데이터 유지!

자, 화면 공유하면서 같이 해볼게요!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

이것만 기억하세요!

오늘의 핵심 키워드 3가지

BaaS와 Firebase

서버 관리 불필요!
구글이 알아서 해줘요

Firestore 구조

컬렉션(폴더) - 문서(파일)
- 서브컬렉션(폴더 속 폴더)

새로고침 문제 해결!

메모리 저장 →
Firestore 영구 저장

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘 배운 것

이론

BaaS 개념
Backend as a Service, 서버 관리 불필요

Firebase 소개
구글 제공 BaaS 플랫폼

Firestore 구조
컬렉션, 문서, 서브컬렉션

마이링크 설계
users/anonymous/links/{linkId}

실습 완료

- Firebase 프로젝트 생성
- Firestore 데이터베이스 활성화
- Firebase SDK 설치 및 Next.js 연동
- 링크 저장(Create) 및 불러오기(Read)
- 새로고침해도 데이터 유지! 🎉

아이디어를 현실로 - **바이브 코딩**

THANK YOU